DIAGNOSTIC REPORT

Cat in the Garden: why the planet is slow and blurry up close

A read of the source and the 15 capture frames. No changes made, this is just the findings. Target hardware in the captures is a Quadro M1000M (Maxwell, ~2015, a weak laptop GPU), running the wgpu / Vulkan backend at 1920x1080 with vsync on.

The short version

There are two separate problems and they have two separate causes. They are not really fighting each other, they just both get worse as you descend.

1. It looks bad up close because the terrain data runs out.

The whole planet is generated from a single 4096 x 2048 height field on a 4,000 km radius world. That is roughly 6 km per height sample at the equator. Real per-tile data is only written down to quadtree level 4. Below that, nearly everything falls back to a stretched coarse ancestor tile. So when you fly down to 3 km altitude you are looking at kilometre-scale samples blown up, with the fine geometric detail layer switched off in code. The result is smooth green mounds with no crisp features anywhere except one hand-picked landing spot.

2. It runs slow up close because it is fill-bound, not geometry-bound.

Draw counts are tiny (about 60 to 130 per frame) and triangle counts are modest (under 600k). Neither of those explains single-digit FPS. What kills it is fragment work: a heavy per-pixel shader drawn over deep, overlapping terrain, with no depth pre-pass, no front-to-back sorting, and a discard in the fragment shader that switches off early depth rejection. So on a slow GPU every layer of hill behind hill still gets fully shaded.

What the 15 frames show

Numbers read straight off the debug overlay in each capture. Two columns matter most: fallback chunks (how much of the view is stretched coarse data) and FPS. Watch how both track altitude.

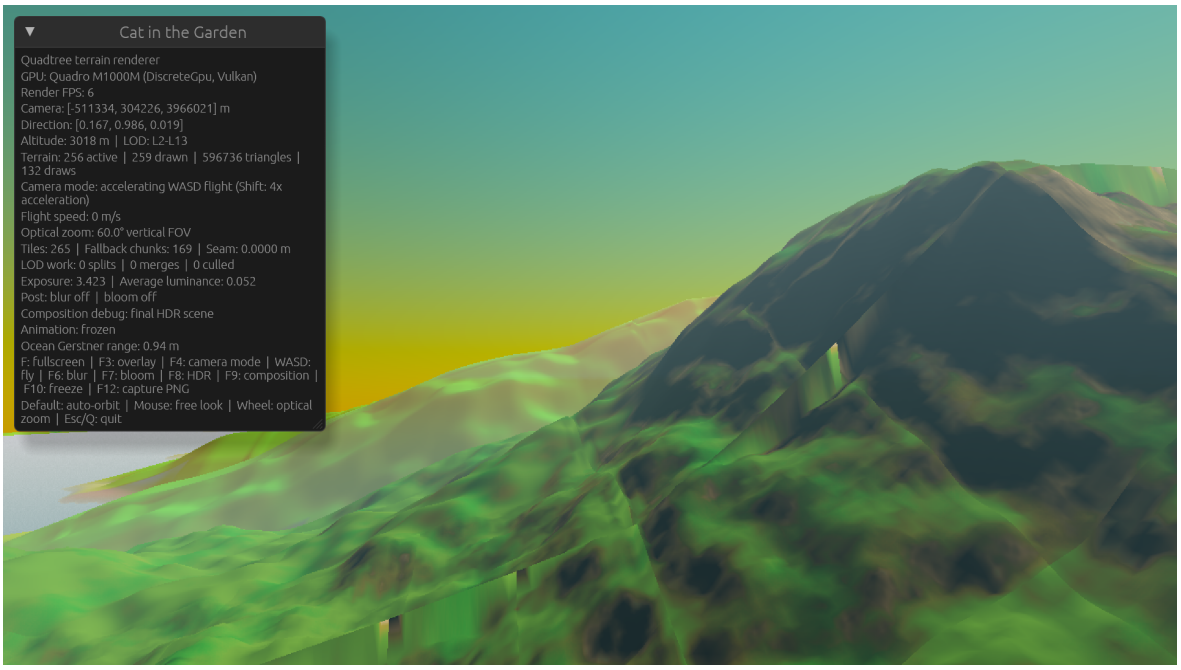
#	Altitude	FPS	LOD	Triangles	Draws	Fallback chunks
1	3,018 m	9	L4-L13	474,624	124	113
2	3,018 m	1	L4-L13	474,624	124	113
3	3,018 m	26	L2-L13	262,656	80	55
4	3,018 m	6	L2-L13	596,736	132	169
5	3,018 m	10	L2-L13	400,896	82	115
6	4,019 m	7	L2-L13	474,624	133	103
7	9,050 m	7	L2-L12	446,976	108	121
8	16,707 m	11	L2-L11	421,632	76	127
9	34,468 m	10	L2-L10	419,328	61	142
10	135,969 m	13	L2-L8	336,384	60	107
11	271,880 m	10	L2-L7	315,648	59	81
12	612,878 m	19	L2-L6	288,000	68	59
13	1,391,235 m	27	L2-L5	255,744	85	14
14	3,359,085 m	59	L2-L3	165,888	72	0
15	6,087,646 m	59	L2	87,552	38	0

Frames 1 to 9 are low, single to low-double-digit FPS, 100+ fallback chunks. Frames 14 and 15 are from orbit, locked at 59 (the vsync ceiling) with zero fallback. The trend is the whole story.

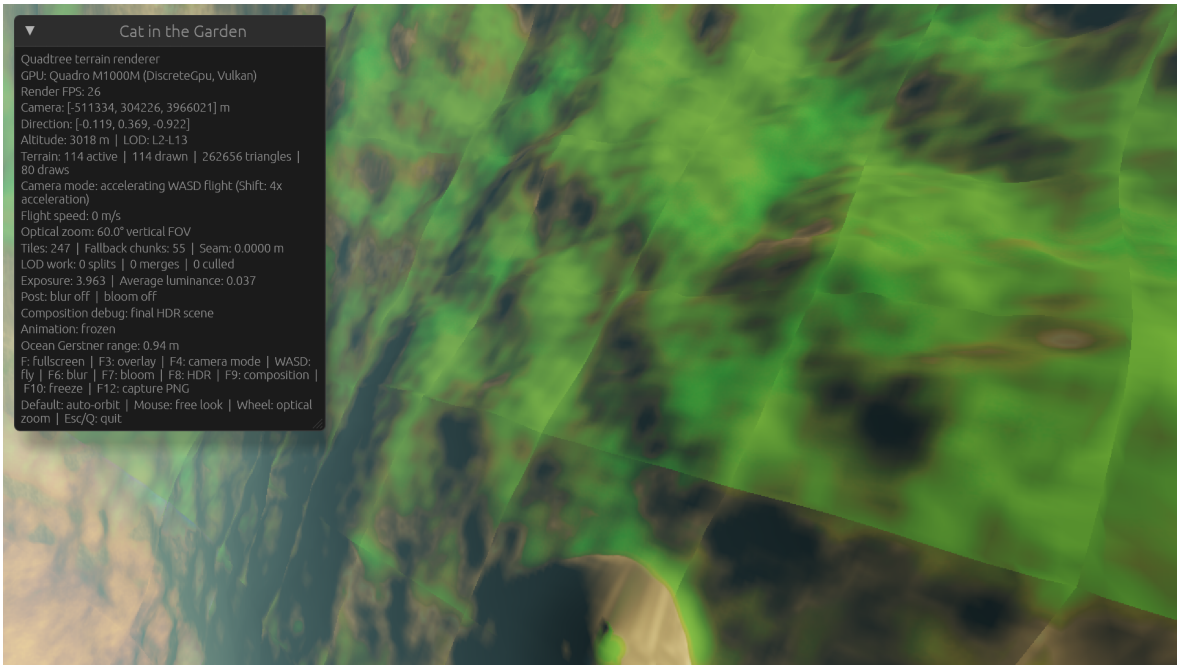
The clincher: frames 3 and 4, same altitude, different look direction.

Both are at 3,018 m. Frame 3 points down at the ground and gets 26 FPS. Frame 4 points at the horizon and drops to 6 FPS. Same camera height, same shader, same data. The only thing that changed is how many layers of terrain stack up behind each other on screen. That is a textbook overdraw signature. It says the cost is fragments, not triangles or draw calls.

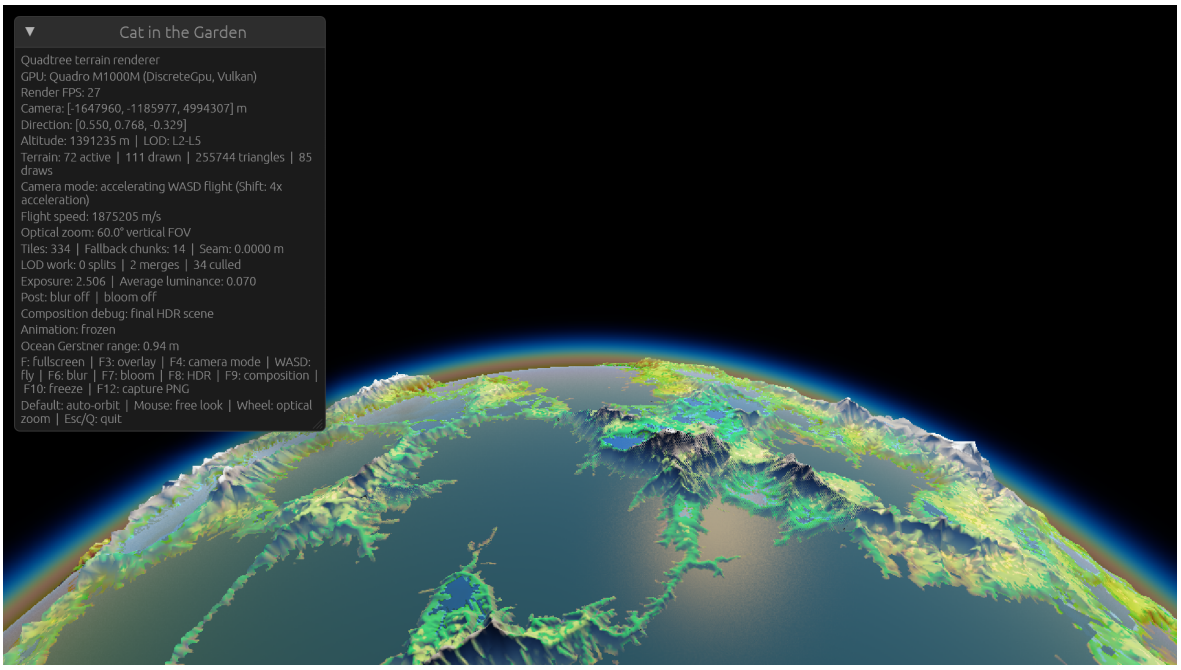
Frames side by side



Frame 4. Altitude 3 km, 6 FPS, 169 fallback chunks. Smooth exaggerated green bulges, no fine relief. This is coarse data upsampled, plus a large vertical exaggeration, plus the runtime detail layer turned off. The soft look is baked in, not a blur filter.



Frame 3. Same 3 km altitude but looking straight down, 26 FPS. Fewer terrain layers overlap, so less overdraw, so it runs four times faster. Same scene, cheaper because of geometry on screen.



Frame 13. From ~1,400 km up. Fallback has dropped to 14 and FPS climbs to 27. Note the blocky, stair-stepped coastlines. That is the coarse tile and sample grid showing through as you approach the data ceiling from above.

Why it looks bad (in detail)

The data ceiling

In the baker config the working grid is 4096 x 2048 and dense_level is 4. The base shape comes from a handful of Perlin octaves (fbm with 6 octaves, ridged fbm with 5) evaluated on that grid, then thermal erosion, rivers, lakes and glacial carving on the same grid. Everything the planet knows about its own surface lives at that resolution. On a 4,000 km radius planet the equatorial circumference is about 25,000 km, so 4096 samples across is roughly 6 km per sample.

There is no finer signal anywhere. The sparse refinement past level 4 samples the exact same smooth field at a higher tile rate, so it interpolates, it does not invent detail. That is why even the one detailed landing area still reads as soft.

Fallback upsampling is doing most of the near-ground work

resolve_tile walks up to best_available_ancestor when the requested tile is not baked, and the renderer draws that ancestor stretched across the child with a UV transform. The overlay counts this as a fallback chunk. At 3 km you see 100 to 170 fallback chunks out of ~200 active. So more than half of what is on screen near the ground is one coarse tile smeared over a wide area.

Vertical exaggeration plus no runtime relief

Two constants shape the look. OUTMAP_TERRAIN_NEAR_HEIGHT_SCALE is 9.0 and the far value is 36.0, blended by altitude, so heights are pushed up 9x near the ground. And GLOBAL_TERRAIN_DETAIL_HEIGHT_SCALE is 0.0, which means the high-frequency detail octaves that could add crunch to fallback tiles are applied to colour only, never to geometry. The comment in the code says the physical-amplitude version read as an endless field of cones, so it was zeroed. The net effect is tall smooth swells with no small-scale shape. That is exactly what the captures show.

LOD popping and blocky coasts

At mid altitude (frames 10 to 13) the coastlines are stair-stepped. That is the coarse tile and sample grid quantising the shoreline. It is the same root cause: not enough resolution to place an edge smoothly.

Why it runs slow at low altitude (in detail)

The rendering path is a single geometry pass. The atmosphere draws as a fullscreen quad first, then terrain draws on top. There is no depth pre-pass. That combination has a few compounding costs, and they only bite when the terrain fills the screen with depth, which is exactly low flight.

- **Discard defeats early-Z.** fs_main runs a dither discard for LOD cross-fades. On most GPUs the mere presence of discard in a shader disables early depth rejection for the whole draw, so occluded fragments still run the full shader before being thrown away.
- **No front-to-back ordering.** Terrain is sorted by source tile key so it can batch instances. Good for draw count, but it means near hills are not drawn before far hills, so even where depth testing could help it often does not.
- **No depth pre-pass.** A cheap depth-only pass would let the expensive colour pass reject hidden fragments up front. Without it, and with the two points above, low-altitude depth complexity turns into near-full overdraw of a heavy shader.
- **Coastline fragments pay twice.** Any land fragment with partial ocean coverage runs the full terrain material path and the full ocean path in the same shader: six Gerstner waves, ocean lighting, sky diffuse and aerial perspective. Almost every capture is coast, so this is the common case, not the rare one.
- **Fallback chunks cost full price for no benefit.** The screen-space split ratio for the outmap path is 0.15, which is aggressive, so it keeps splitting down to the level limit even where the source is only level 4. Those extra chunks add fragment and vertex work while showing zero new detail, because they all sample the same coarse ancestor.

From orbit none of this matters. Depth complexity is about 1, the planet is convex and mostly front-facing, fallback is zero, and it simply hits vsync at 59. That is why the fast case and the good-looking case are the same case.

Worth saying: a lot of this is well built

This is not a mess. The bones are solid, which is part of why the two weak spots stand out.

- Tile loading runs on a worker thread with a bounded upload budget and an LRU-style cache, so disk I/O is not stalling the frame.
- Auto-exposure reads back luminance through an async ring buffer, no blocking map on the render thread.
- Chunks share one canonical mesh and draw instanced, so draw calls stay low even with hundreds of chunks.
- Seams, skirts (capped at 10 m, so not an overdraw problem), LOD cross-fades and an adjacent-level delta clamp are all handled deliberately.
- The atmosphere, HDR and tone-mapping pipeline is real and physically motivated, and it shows in the orbit and sunset frames.

If and when you want to act, here is the priority order

Not doing any of this yet, per your ask. This is just where the leverage is, biggest first.

1	**Fix the data ceiling for the look** Nothing on the render side can add detail that the data does not have. Either bake a much higher resolution field, or add real runtime detail that survives up close. The runtime octave layer already exists and is set to zero height. Turning it on with a screen-stable, seam-safe amount is the cheapest path to crisp ground. A higher dense_level or a proper detail-noise pass is the thorough path.
2	**Add a depth pre-pass and shade front-to-back** A depth-only pass, then colour, is the standard fix for fill-bound terrain. It lets the heavy shader skip hidden fragments. Pairing that with a rough front-to-back draw order for the colour pass recovers most of the low-altitude cost.
3	**Get discard out of the hot path** The LOD dither discard disables early-Z everywhere. Options include doing the cross-fade without discard, or keeping discard only for the short window when a fade is actually running rather than on every frame for every chunk.
4	**Stop over-splitting into fallback** Cap the split so it does not refine past the level where real data exists off the landing site. Those chunks currently cost full price and show nothing. This helps both FPS and, honestly, the look, since fewer stretched tiles is less mush.
5	**Split the coastline double-shading** Land, ocean and the thin coast band do not all need the full ocean path. Branch or mask so open land does not evaluate six Gerstner waves it will not use.

One line to take away: the planet is slow and blurry near the ground for the same underlying reason. It is spending a lot of expensive fragment work drawing coarse data it does not have the detail to justify. Give it detail, and stop paying for hidden and redundant fragments, and both problems move together.